

第 30 讲核心概念速查卡：nn.Module

1. 最小模板

```
class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 4)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(4, 2)

    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))
```

2. 两个核心职责

位置	职责
<code>__init__()</code>	注册 Layer / Parameter / Buffer，描述“模型有哪些零件”。
<code>forward()</code>	描述 Tensor 怎样经过这些零件，定义“数据流”。

3. Parameter Shape 与数量

对于 $3 \rightarrow 4 \rightarrow 2$:

$fc1.weight : (4, 3), \quad fc1.bias : (4,)$

$fc2.weight : (2, 4), \quad fc2.bias : (2,)$

总参数量:

$$3 \times 4 + 4 + 4 \times 2 + 2 = 26$$

4. 常用检查接口

```
print(model)
model.parameters()
model.named_parameters()
model.named_buffers()
model.state_dict()
model.to(device)
model.zero_grad()
```

5. Parameter / Buffer / 普通 Tensor

类型	进入 parameters	进入 state_dict	随 model.to 迁移
Parameter	是	是	是
Buffer	否	是	是
普通 Tensor 属性	否	否	不自动保证

6. 保存/加载

```
torch.save(model.state_dict(), path)
model2 = TinyMLP()
model2.load_state_dict(torch.load(path))
```

state_dict 是状态，不是完整 Python 模型定义；加载前通常仍需要兼容的模型结构。

7. 调用习惯

推荐：

```
y = model(x)
```

而不是日常直接写：

```
y = model.forward(x)
```

因为 `model(x)` 会经过 Module 的 `__call__` 机制，再进入 `forward()`。

8. 与 Transformer 的关系

大型 Transformer 可以理解为：

Module 树 = Embedding + $N \times$ Block + LM Head

每个 Block 又包含 Attention、Norm、MLP 等子 Module。

读 PyTorch 模型源码的固定顺序：先 `__init__()` 看零件和参数，再 `forward()` 看 Tensor 与 Shape。